COP509_Notebook1_Search

May 20, 2026

1 COP509: Document Analysis System

1.1 Notebook 1: Search System Student ID: F231130

Module: COP509 Natural Language Processing | Loughborough University

---

1.1.1 Overview

This notebook builds a search system over 12 government inquiry documents — 6 recommendation/response pairs. Three methods are implemented and compared: TF-IDF cosine similarity as a keyword baseline, semantic search using sentence transformer embeddings, and a hybrid approach that combines both scores.

The documents cover Space Economy, UK Covid-19 Module 1, Behaviour Change, Post Office Horizon IT, Infected Blood Inquiry, and WASPI.

1.2 Environment Setup

`pdfplumber` for layout-aware PDF extraction, `sentence-transformers` for semantic search, `scikit-learn` for TF-IDF and cosine similarity, `ipywidgets` for the search interface.

[1]:
```
# Install required libraries
# run at the start of each collab session

!pip install pdfplumber sentence-transformers scikit-learn ipywidgets openpyxl␣
↪--quiet

print(" All libraries installed")
```

```
All libraries installed
```

1.3 Loading the Dataset

12 PDFs in 6 recommendation/response pairs, plus 6 ground truth Excel files. All stored in Google Drive and loaded directly into Colab.

[2]:
```
# Mount Google Drive and verify all dataset files exist
```

```python
from google.colab import drive
drive.mount('/content/drive')

import os

#   the exact dataset path
BASE_PATH = "/content/drive/My Drive/Colab Notebooks/21COP509/LabDatasets/
 ↪Datasetsfiles"

# Verify the path exists
if not os.path.exists(BASE_PATH):
    raise FileNotFoundError(f"Dataset path not found: {BASE_PATH}\n"
                            f"Check your Drive is mounted and the path is␣
 ↪correct.")

# List all files found at that path
all_files = os.listdir(BASE_PATH)
print(f" Found {len(all_files)} files at dataset path:\n")
for f in sorted(all_files):
    size_kb = os.path.getsize(os.path.join(BASE_PATH, f)) / 1024
    print(f"  {f:<70} {size_kb:>8.1f} KB")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
 Found 21 files at dataset path:

  .DS_Store                                                              
10.0 KB
  Behaviour-Change-Report-Recomm.pdf                                     
760.1 KB
  Behaviour-Change-Response.pdf                                          
222.1 KB
  Behaviour_Change_Recommendations_Responses.xlsx                        
23.2 KB
  Dataset_Characteristics_Summary.xlsx                                   
7.3 KB
  PostOfficeHorizon-I- Inquiry-Recomm.pdf                                
1149.4 KB
  PostOfficeHorizon-IT-Inquiry-Response.pdf                              
3422.7 KB
  PostOffice_Horizon_Recommendations_Responses.xlsx                      
12.8 KB
  Space_Economy_Recommendations_Responses.xlsx                           
29.5 KB
  TheSpaceEconomyReport.pdf                                              
2897.3 KB
  TheSpaceEconomyResponse.pdf
```

```
986.8 KB
  UK-Covid-19-Inquiry-Module-1-Recomm.pdf
1941.7 KB
  UK-Covid-19_Inquiry_Module_1_Response.pdf
286.8 KB
  UK_Covid19_Inquiry_Module1_Recommendations_Responses.xlsx
11.7 KB
  Volume_1-Blood-Inquiry-Recomm.pdf
2371.8 KB
  Volume_1-Blood-Inquiry-Response.pdf
456.6 KB
  Volume_I-BloodEnquiry_Recommendations_Responses.xlsx
17.7 KB
  WASPI_Recommendations_Recommendations_Responses.xlsx
7.0 KB
  governments_new_decision_womens_state_pension_age_communications-Recomm.pdf
415.3 KB
  governments_new_decision_womens_state_pension_age_communications-Response.pdf
415.3 KB
  ~$Dataset_Characteristics_Summary.xlsx
0.2 KB
```

1.4 Dataset Registry

All file paths and dataset metadata are kept in one dictionary rather than scattered through the code. Easier to update and ensures every function references the same names and paths.

[3]:
```python
#  Master dataset registry
# Maps each dataset to its PDF pair, ground truth XLSX,
# extraction strategy, and known recommendation count.
# This registry is used by every subsequent function.

def p(filename):
    """Helper: builds full path from filename"""
    return os.path.join(BASE_PATH, filename)

DATASETS = {
    "space_economy": {
        "rec_pdf":      p("TheSpaceEconomyReport.pdf"),
        "res_pdf":      p("TheSpaceEconomyResponse.pdf"),
        "ground_truth": p("Space_Economy_Recommendations_Responses.xlsx"),
        "extraction":   "structural",
        "total_recs":   40,
        "label":        "Space Economy"
    },
    "covid19": {
        "rec_pdf":      p("UK-Covid-19-Inquiry-Module-1-Recomm.pdf"),
        "res_pdf":      p("UK-Covid-19_Inquiry_Module_1_Response.pdf"),
```

```
        "ground_truth": p("UK_Covid19_Inquiry_Module1_Recommendations_Responses.
↪xlsx"),
        "extraction":   "structural",
        "total_recs":   10,
        "label":        "UK Covid-19 Module 1"
    },
    "behaviour_change": {
        "rec_pdf":      p("Behaviour-Change-Report-Recomm.pdf"),
        "res_pdf":      p("Behaviour-Change-Response.pdf"),
        "ground_truth": p("Behaviour_Change_Recommendations_Responses.xlsx"),
        "extraction":   "structural",
        "total_recs":   33,
        "label":        "Behaviour Change"
    },
    "post_office": {
        "rec_pdf":      p("PostOfficeHorizon-I- Inquiry-Recomm.pdf"),
        "res_pdf":      p("PostOfficeHorizon-IT-Inquiry-Response.pdf"),
        "ground_truth": p("PostOffice_Horizon_Recommendations_Responses.xlsx"),
        "extraction":   "numbered",
        "total_recs":   19,
        "label":        "Post Office Horizon IT"
    },
    "infected_blood": {
        "rec_pdf":      p("Volume_1-Blood-Inquiry-Recomm.pdf"),
        "res_pdf":      p("Volume_1-Blood-Inquiry-Response.pdf"),
        "ground_truth": p("Volume_I-BloodEnquiry_Recommendations_Responses.
↪xlsx"),
        "extraction":   "structural",
        "total_recs":   58,
        "label":        "Infected Blood Inquiry"
    },
    "waspi": {
        "rec_pdf":      ␣
↪p("governments_new_decision_womens_state_pension_age_communications-Recomm.
↪pdf"),
        "res_pdf":      ␣
↪p("governments_new_decision_womens_state_pension_age_communications-Response.
↪pdf"),
        "ground_truth": p("WASPI_Recommendations_Recommendations_Responses.
↪xlsx"),
        "extraction":   "embedded",
        "total_recs":   3,
        "label":        "WASPI"
    }
}

# Verify every file exists before proceeding
```

```python
print("Verifying all dataset files...\n")
all_ok = True
for name, info in DATASETS.items():
    for key in ["rec_pdf", "res_pdf", "ground_truth"]:
        exists = os.path.exists(info[key])
        status = "EXISTS" if exists else "MISSING"
        if not exists:
            all_ok = False
        print(f"  [{status}] {name} → {key}: {os.path.basename(info[key])}")

print()
print("All files verified - ready to proceed." if all_ok
      else "Fix missing files before continuing.")
```

```
Verifying all dataset files…

  [EXISTS] space_economy → rec_pdf: TheSpaceEconomyReport.pdf
  [EXISTS] space_economy → res_pdf: TheSpaceEconomyResponse.pdf
  [EXISTS] space_economy → ground_truth:
Space_Economy_Recommendations_Responses.xlsx
  [EXISTS] covid19 → rec_pdf: UK-Covid-19-Inquiry-Module-1-Recomm.pdf
  [EXISTS] covid19 → res_pdf: UK-Covid-19_Inquiry_Module_1_Response.pdf
  [EXISTS] covid19 → ground_truth:
UK_Covid19_Inquiry_Module1_Recommendations_Responses.xlsx
  [EXISTS] behaviour_change → rec_pdf: Behaviour-Change-Report-Recomm.pdf
  [EXISTS] behaviour_change → res_pdf: Behaviour-Change-Response.pdf
  [EXISTS] behaviour_change → ground_truth:
Behaviour_Change_Recommendations_Responses.xlsx
  [EXISTS] post_office → rec_pdf: PostOfficeHorizon-I- Inquiry-Recomm.pdf
  [EXISTS] post_office → res_pdf: PostOfficeHorizon-IT-Inquiry-Response.pdf
  [EXISTS] post_office → ground_truth:
PostOffice_Horizon_Recommendations_Responses.xlsx
  [EXISTS] infected_blood → rec_pdf: Volume_1-Blood-Inquiry-Recomm.pdf
  [EXISTS] infected_blood → res_pdf: Volume_1-Blood-Inquiry-Response.pdf
  [EXISTS] infected_blood → ground_truth: Volume_I-
BloodEnquiry_Recommendations_Responses.xlsx
  [EXISTS] waspi → rec_pdf:
governments_new_decision_womens_state_pension_age_communications-Recomm.pdf
  [EXISTS] waspi → res_pdf:
governments_new_decision_womens_state_pension_age_communications-Response.pdf
  [EXISTS] waspi → ground_truth:
WASPI_Recommendations_Recommendations_Responses.xlsx

All files verified - ready to proceed.
```

1.5 Ground Truth Files

The Excel files contain manually verified recommendation text, response text and classification labels. Loading them upfront means the system output can be checked against known correct answers at any point, not just at the end.

[4]:
```
# Load all ground truth Excel files
# These are loaded NOW and kept in memory throughout both
# notebooks. They contain exact recommendation text, response
# text, and classification labels - used for validation
# throughout development, not just at evaluation time.


import pandas as pd

ground_truths = {}

for name, info in DATASETS.items():
    try:
        df = pd.read_excel(info["ground_truth"])
        ground_truths[name] = df
        print(f" {info['label']:<30} {len(df):>3} rows | "
              f"Columns: {list(df.columns)}")
    except Exception as e:
        print(f" {name}: Failed to load - {e}")

print(f"\nTotal ground truth records: "
      f"{sum(len(df) for df in ground_truths.values())}")
```

```
 Space Economy                  40 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
Development']
 UK Covid-19 Module 1           10 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
Development']
 Behaviour Change               33 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
Development']
 Post Office Horizon IT         19 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
```

```
Development']
 Infected Blood Inquiry          58 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
Development']
 WASPI                            3 rows | Columns: ['Rec File Name', 'Rec File
ID', 'Rec File Title', 'Rec ID', 'Rec Extracted', 'Res File Name', 'Res File
ID', 'Res File Title', 'Res ID', 'Res Extracted', 'Status
(Accept/Reject/Partial)', 'Best Extraction Approach', 'Tips for Code
Development']

Total ground truth records: 163
```

1.6 Step 1: PDF Text Extraction

`pdfplumber` handles multi-column layouts and preserves reading order more reliably than simpler PDF readers. Each page is extracted and stored with its dataset name, document type, and page number so results can be traced back to their source.

[5]:
```python
import pdfplumber

def extract_pdf_pages(pdf_path, dataset_name, doc_type):
    pages   = []
    skipped = []

    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages):
            text = page.extract_text()

            if text and len(text.strip()) > 50:
                pages.append({
                    "dataset":  dataset_name,
                    "doc_type": doc_type,
                    "pdf_path": pdf_path,
                    "page":     i + 1,
                    "text":     text.strip()
                })
            else:
                skipped.append(i + 1)

    if skipped:
        print(f"  [skipped pages] {os.path.basename(pdf_path)}: {skipped}")

    return pages


print("Extracting text from all PDFs...\n")
```

```python
all_pages = []

for name, info in DATASETS.items():
    for doc_type, pdf_key in [("recommendation", "rec_pdf"),
                              ("response",       "res_pdf")]:
        pages = extract_pdf_pages(info[pdf_key], name, doc_type)
        all_pages.extend(pages)
        print(f"  {info['label']:<30} [{doc_type:<14}] {len(pages):>3} pages")

pages_df = pd.DataFrame(all_pages)
print(f"\nTotal pages extracted: {len(pages_df)}")
print("\nBreakdown by dataset and document type:")
print(pages_df.groupby(["dataset", "doc_type"]).size()
              .rename("pages").reset_index().to_string(index=False))
```

```
Extracting text from all PDFs…

  [skipped pages] TheSpaceEconomyReport.pdf: [8, 134]
  Space Economy                  [recommendation] 132 pages
  [skipped pages] TheSpaceEconomyResponse.pdf: [2, 34, 35, 36]
  Space Economy                  [response      ]  32 pages
  [skipped pages] UK-Covid-19-Inquiry-Module-1-Recomm.pdf: [2, 15, 18, 19, 71,
94, 120, 151, 197, 210, 236, 237, 238, 239, 240]
  UK Covid-19 Module 1           [recommendation] 225 pages
  [skipped pages] UK-Covid-19_Inquiry_Module_1_Response.pdf: [2, 39]
  UK Covid-19 Module 1           [response      ]  38 pages
  [skipped pages] Behaviour-Change-Report-Recomm.pdf: [6]
  Behaviour Change               [recommendation] 110 pages
  Behaviour Change               [response      ]  19 pages
  [skipped pages] PostOfficeHorizon-I- Inquiry-Recomm.pdf: [2, 165]
  Post Office Horizon IT         [recommendation] 164 pages
  [skipped pages] PostOfficeHorizon-IT-Inquiry-Response.pdf: [1, 2, 3, 4, 5, 6,
7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
  Post Office Horizon IT         [response      ]   0 pages
  Infected Blood Inquiry         [recommendation] 295 pages
  [skipped pages] Volume_1-Blood-Inquiry-Response.pdf: [2, 29, 68]
  Infected Blood Inquiry         [response      ]  65 pages
  WASPI                          [recommendation]  49 pages
  WASPI                          [response      ]  49 pages

Total pages extracted: 1178

Breakdown by dataset and document type:
         dataset       doc_type  pages
behaviour_change recommendation    110
behaviour_change       response     19
         covid19 recommendation    225
```

```
        covid19       response     38
infected_blood recommendation    295
infected_blood       response     65
   post_office recommendation    164
 space_economy recommendation    132
 space_economy       response     32
         waspi recommendation     49
         waspi       response     49
```

[6]:
```python
import nltk
import re
from nltk.corpus import stopwords

nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)

stop_words = set(stopwords.words('english'))

def preprocess_text(text, for_search=True):
    """
    Cleans text for search indexing.
    for_search=True applies full cleaning for TF-IDF/BM25.
    for_search=False applies lighter cleaning for semantic search
    (semantic models handle punctuation better than keyword models).
    """
    # lowercase
    text = text.lower()
    # remove page numbers and artefacts
    text = re.sub(r'\b\d{1,3}\b', '', text)
    # normalise whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    if for_search:
        # remove punctuation
        text = re.sub(r'[^\w\s]', '', text)
        # remove stopwords
        tokens = text.split()
        tokens = [t for t in tokens if t not in stop_words]
        text = ' '.join(tokens)

    return text
```

Chunk size follows the RAG tutorial's guidance of 500-1000 characters with around 10% overlap as a starting point. We use 200 words (~1,200 characters) with 50-word overlap (25%) — higher overlap than recommended because government inquiry paragraphs are dense and recommendations frequently straddle chunk boundaries.

[7]:
```python
# Sliding window chunking + build master chunk store

def chunk_sliding_window(text, chunk_size=200, overlap=50):
    """
    Split text into overlapping word-level windows.

    chunk_size: number of words per chunk
    overlap   : words shared between consecutive chunks

    Returns list of chunk strings.
    """
    words  = text.split()
    chunks = []
    start  = 0

    while start < len(words):
        end   = start + chunk_size
        chunk = " ".join(words[start:end])
        if len(chunk.strip()) > 20:
            chunks.append(chunk)
        start += chunk_size - overlap

    return chunks


# Build the master chunk store from all extracted pages
print("Building chunk store...\n")
all_chunks = []

for _, page_row in pages_df.iterrows():
    chunks = chunk_sliding_window(page_row["text"])
    for j, chunk in enumerate(chunks):
        all_chunks.append({
            "dataset":     page_row["dataset"],
            "doc_type":    page_row["doc_type"],
            "page":        page_row["page"],
            "chunk_index": j,
            "chunk_id":    (f"{page_row['dataset']}_"
                            f"{page_row['doc_type']}_"
                            f"p{page_row['page']}_c{j}"),
            "text":        chunk
        })

chunk_store = pd.DataFrame(all_chunks)
chunk_store = chunk_store.reset_index(drop=True)

print(f"Total chunks: {len(chunk_store)}\n")
```

```
print("Chunks per dataset:")
print(chunk_store.groupby(["dataset", "doc_type"])
                 .size()
                 .rename("chunks")
                 .reset_index()
                 .to_string(index=False))
```

```
Building chunk store…

Total chunks: 3922

Chunks per dataset:
         dataset       doc_type  chunks
behaviour_change recommendation     340
behaviour_change       response      58
         covid19 recommendation     694
         covid19       response      95
  infected_blood recommendation    1099
  infected_blood       response     149
     post_office recommendation     637
   space_economy recommendation     463
   space_economy       response      87
           waspi recommendation     150
           waspi       response     150
```

3,922 chunks across 11 documents. The Post Office response is missing — pdfplumber returned zero pages from that PDF. That is addressed in Challenge 2 below. Everything else has chunks for both recommendation and response types.

[8]:
```
# Post Office response - fallback to ground truth XLSX
# The PostOfficeHorizon-IT-Inquiry-Response.pdf is entirely
# image-based (scanned document). pdfplumber extracts 0 pages.
# For Task 2 alignment, I load response text from the ground
# truth XLSX directly, which contains pre-extracted text.
# For Task 1 search I attempt OCR below (see Challenge 2).

po_gt = ground_truths["post_office"]

# Build a synthetic response dataframe from the XLSX
po_responses_from_gt = pd.DataFrame({
    "dataset":   "post_office",
    "doc_type":  "response",
    "page":      po_gt.index + 1,   # synthetic page numbers
    "chunk_index": 0,
    "chunk_id":  ["post_office_response_gt_" + str(i)
                  for i in po_gt.index],
    "text":      po_gt["Res Extracted"].dropna()
})
```

```
print(f"Post Office responses loaded from XLSX: "
      f"{len(po_responses_from_gt)} rows")
display(po_responses_from_gt.head(3))
```

```
Post Office responses loaded from XLSX: 19 rows
```

	dataset	doc_type	page	chunk_index	chunk_id	text
0	post_office	response	1	0	post_office_response_gt_0	DBT accepts this recommendation and has fulfil…
1	post_office	response	2	0	post_office_response_gt_1	DBT accepts the Inquiry's recommendation that …
2	post_office	response	3	0	post_office_response_gt_2	DBT accepts this recommendation. Both DBT and …

[9]:
```
# Challenge 2 - OCR for image-based PDF
# (PostOfficeHorizon-IT-Inquiry-Response.pdf)

# Install OCR dependencies - only needs to run once per session

!apt-get install -y poppler-utils tesseract-ocr --quiet
!pip install pytesseract pdf2image --quiet

from pdf2image import convert_from_path
import pytesseract

OCR_OUTPUT_PATH = (
    "/content/drive/My Drive/Colab Notebooks/21COP509/"
    "PostOffice_Response_OCR.txt"
)

PO_RES_PDF = DATASETS["post_office"]["res_pdf"]

# Only run OCR if we haven't already cached the result
if os.path.exists(OCR_OUTPUT_PATH):
    with open(OCR_OUTPUT_PATH, "r") as f:
        po_ocr_text = f.read()
    print(f" Loaded cached OCR text ({len(po_ocr_text):,} chars)")

else:
    print("Running OCR - this takes 3-5 minutes for 16 pages...")
    images = convert_from_path(PO_RES_PDF, dpi=300)

    all_text = []
    for i, img in enumerate(images):
        text = pytesseract.image_to_string(img, lang="eng")
```

```
        all_text.append(f"\n\n--- Page {i+1} ---\n\n{text}")
        print(f"  Page {i+1}/{len(images)} done")

    po_ocr_text = "\n".join(all_text)

    # Save to Drive so we never have to run this again
    with open(OCR_OUTPUT_PATH, "w") as f:
        f.write(po_ocr_text)

    print(f"\n OCR complete - {len(po_ocr_text):,} chars extracted "
          f"and cached to Drive")

# Add OCR'd chunks to chunk store
po_ocr_chunks = chunk_sliding_window(po_ocr_text)
po_ocr_rows = [{
    "dataset":     "post_office",
    "doc_type":    "response",
    "page":        0,          # 0 = OCR source
    "chunk_index": j,
    "chunk_id":    f"post_office_response_ocr_c{j}",
    "text":        chunk
} for j, chunk in enumerate(po_ocr_chunks)]

# Append to chunk store and rebuild indexes
new_rows    = pd.DataFrame(po_ocr_rows)
chunk_store = pd.concat([chunk_store, new_rows], ignore_index=True)

print(f"\n Added {len(po_ocr_rows)} OCR chunks to chunk store")
print(f"  New total chunks: {len(chunk_store)}")
```

```
Reading package lists…
Building dependency tree…
Reading state information…
tesseract-ocr is already the newest version (4.1.1-2.1build1).
poppler-utils is already the newest version (22.02.0-2ubuntu0.12).
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
 Loaded cached OCR text (27,295 chars)

 Added 27 OCR chunks to chunk store
  New total chunks: 3949
```

1.6.1 Chunking Strategy

The RAG tutorial recommends 500-1000 characters with 10% overlap. We use 200 word. (~1,200 characters) with 50-word overlap (25%) for two reasons specific to this document type.

Government recommendations typically run 150-250 words. A 200-word chunk keeps a single recommendation intact rather than splitting it across two chunks. The higher overlap — 25%

rather than the recommended 10% — ensures sentences near chunk boundaries appear fully in at least one chunk rather than being cut off.

The chunk store contains 3,949 chunks across 12 documents. Post Office contributes only 27 due to OCR noise from the webpage-printed source, compared to 87-694 for other documents. That disparity is documented in Challenge 2.

[24]:
```
# Validate chunk store coverage after OCR fix

coverage = chunk_store.groupby(["dataset","doc_type"]).size().reset_index()
coverage.columns = ["dataset", "doc_type", "chunks"]
coverage["status"] = coverage["chunks"].apply(
    lambda n: " OK" if n > 0 else "[EMPTY] - check extraction"
)

print("Chunk store coverage check:\n")
print(coverage.to_string(index=False))

# Flag any empty combinations
empty = coverage[coverage["chunks"] == 0]
if empty.empty:
    print("\n All datasets have chunks for both doc types")
else:
    print(f"\n Still empty: {empty[['dataset','doc_type']].values}")
```

```
Chunk store coverage check:

         dataset       doc_type  chunks status
behaviour_change recommendation     340     OK
behaviour_change       response      58     OK
         covid19 recommendation     694     OK
         covid19       response      95     OK
  infected_blood recommendation    1099     OK
  infected_blood       response     149     OK
     post_office recommendation     637     OK
     post_office       response      27     OK
   space_economy recommendation     463     OK
   space_economy       response      87     OK
           waspi recommendation     150     OK
           waspi       response     150     OK

 All datasets have chunks for both doc types
```

27 chunks added, bringing the total to 3,949. The count is low compared to other response documents because OCR output is noisier and more chunks fail the minimum length check. The Post Office response is now searchable alongside the other 11 documents.

These chunks carry `page = 0` to flag them as OCR-sourced rather than directly extracted. This makes it possible to identify them in search results if needed.

1.7 Step 2: Text Preprocessing

Before building the indexes, the extracted text is cleaned. Tutorial 1 makes the point that cleaning is task-specific — what helps one method can hurt another.

TF-IDF gets full preprocessing: lowercase, punctuation removal, stopword removal via NLTK. Stopwords like "the" and "of" would otherwise inflate matches on common words rather than meaningful terms.

Semantic search gets lighter treatment: lowercase and whitespace normalisation only. Sentence transformers are trained on natural language including punctuation and function words — stripping these degrades the contextual representations the model depends on.

[11]:
```
import nltk
import re
from nltk.corpus import stopwords

nltk.download('stopwords', quiet=True)

stop_words = set(stopwords.words('english'))

def preprocess_for_tfidf(text):
    """
    Full preprocessing for TF-IDF indexing.
    Lowercase, remove punctuation, remove stopwords.
    Following Tutorial 1 cleaning pipeline.
    """
    text = text.lower()
    text = re.sub(r'\b\d{1,3}\b', '', text)     # remove standalone page numbers
    text = re.sub(r'[^\w\s]', '', text)          # remove punctuation
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = [t for t in text.split() if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)


def preprocess_for_semantic(text):
    """
    Light preprocessing for semantic search.
    Lowercase and normalise whitespace only.
    Sentence transformers handle natural language better
    when punctuation and stopwords are preserved.
    """
    text = text.lower()
    text = re.sub(r'\s+', ' ', text).strip()
    return text


# build two versions of chunk text
chunk_store["text_tfidf"]    = chunk_store["text"].apply(preprocess_for_tfidf)
```

```
chunk_store["text_semantic"] = chunk_store["text"].
 ↪apply(preprocess_for_semantic)

print(f"Preprocessing complete")
print(f"  Chunks processed: {len(chunk_store)}")
print(f"\nSample - original text:")
print(f"  {chunk_store['text'].iloc[0][:120]}")
print(f"\nSample - TF-IDF version:")
print(f"  {chunk_store['text_tfidf'].iloc[0][:120]}")
print(f"\nSample - semantic version:")
print(f"  {chunk_store['text_semantic'].iloc[0][:120]}")
```

```
Preprocessing complete
  Chunks processed: 3949

Sample - original text:
  HOUSE OF LORDS UK Engagement with Space Committee Report of Session 2024-26
The Space Economy: Act Now or Lose Out Order

Sample - TF-IDF version:
  house lords uk engagement space committee report session 2024 space economy
act lose ordered printed october 2026 publis

Sample - semantic version:
  house of lords uk engagement with space committee report of session 2024-26
the space economy: act now or lose out order
```

1.8 Step 3.1: Building the TF-IDF Search Index

TF-IDF scores each word by how often it appears in a chunk (TF) and how rare it is across all chunks (IDF). Common words like "the" or "government" score low. Distinctive words like "spaceport" or "haemophilia" score high. A query is transformed into the same vector space and the closest chunks are returned by cosine similarity.

Bigrams are included so compound terms like "civil emergency" are treated as a single feature rather than two unrelated words.

Tutorial 4 and Lab 4a/4b use TF-IDF as the baseline for information retrieval. It is fast, interpretable, and gives a clear reference point for comparing the semantic and hybrid methods below.

[12]:
```
#  Build TF-IDF index

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

vectorizer = TfidfVectorizer(
    stop_words  = "english",
    ngram_range = (1, 2),
```

```
    max_df      = 0.85,
    min_df      = 2
)
tfidf_matrix = vectorizer.fit_transform(chunk_store["text_tfidf"])
print(f" TF-IDF index built")
print(f"  Chunks indexed : {tfidf_matrix.shape[0]}")
print(f"  Vocabulary size: {tfidf_matrix.shape[1]:,} terms")
```

```
 TF-IDF index built
  Chunks indexed : 3949
  Vocabulary size: 74,539 terms
```

80,388 terms across 3,976 chunks. The sanity check returns Space Economy results for a space funding query — correct. Scores around 0.2 are normal for TF-IDF on longer documents.

[13]:
```
# TF-IDF search function

def search_tfidf(query, top_k=5, dataset_filter=None, doc_type_filter=None):
    """
    Search the chunk store using TF-IDF cosine similarity.

    Parameters:
        query           : search string
        top_k           : number of results to return
        dataset_filter  : restrict to one dataset e.g. "covid19"
        doc_type_filter : "recommendation" or "response"

    Returns:
        DataFrame of top_k results with scores.
    """
    query_vec = vectorizer.transform([query])
    scores    = cosine_similarity(query_vec, tfidf_matrix).flatten()

    results          = chunk_store.copy()
    results["score"] = scores

    if dataset_filter:
        results = results[results["dataset"] == dataset_filter]
    if doc_type_filter:
        results = results[results["doc_type"] == doc_type_filter]

    results = results[results["score"] > 0]
    return results.nlargest(top_k, "score")[
        ["dataset", "doc_type", "page", "score", "text"]
    ].reset_index(drop=True)


# Sanity check
```

```
print("Sanity check - query: 'space funding budget'\n")
print(search_tfidf("space funding budget", top_k=3).to_string())
```

```
Sanity check - query: 'space funding budget'

         dataset        doc_type  page     score
text
0  space_economy  recommendation    23  0.236406
ability of the UK to be truly competitive in space. The recent announcement of a
merger between the UK Space Agency and the DSIT Space Directorate opens up new
questions about the future organisation of space policy across government. •
Space is misunderstood by the public and by businesses-the space sector is
misunderstood and underestimated, with the public unaware of its essential role
in daily life, and non-space businesses unaware of its potential benefits for
their operations. The efficacy of UK space policy is therefore inhibited as it
is not seen as a political priority. • The UK's space strategy is not focused
enough-the approach of the National Space Strategy was perhaps too ambitious
given the lack of significant funding for space. For the UK to develop its
leadership in space under the current financial constraints, choices must be
made about which areas within each capability to prioritise. However, these
essential choices are not being made. Funding for space in the UK 43. Space
funding is delivered through a variety of governmental sources, including the UK
Space Agency, DSIT, the Ministry of Defence and UKRI. A definitive figure of how
much the Government invests in space does not exist, with
1  space_economy  recommendation    23  0.217980
essential choices are not being made. Funding for space in the UK 43. Space
funding is delivered through a variety of governmental sources, including the UK
Space Agency, DSIT, the Ministry of Defence and UKRI. A definitive figure of how
much the Government invests in space does not exist, with the National Audit
Office Report The National Space Strategy and the role of the UK Space Agency,
published in 2024, finding that that DSIT "does not have a good understanding of
the total government funding for civil space across all public bodies".76
Analysis that has been done on the UK's space spending indicates that it spends
far less on space than its international competitors, with the University of
Strathclyde noting that as a proportion of GDP, as well as in terms of shares of
government expenditure, "the UK spends significantly less than the EU
average".77 Professor Brian Cox, Professor of Particle Physics at the University
of Manchester, told us that the UK "spends something like 0.05% of GDP on space.
[ … ] We are low; we are significantly below France and Italy, for example" as a
percentage of GDP.78 However, witnesses also noted that funding for space 76
National
2  space_economy        response     8  0.200543  A strategy for space
Recommendation 1 We acknowledge that the current fiscal environment makes a
significant uplift in funding for space capability development unlikely. The
Government has recognised space as part of the UK's Critical National
Infrastructure and should prioritise funding for space within the budget
available accordingly. The stability of funding ensured with the current three-
```

year spending review cycle is welcome, but the current spending on space should represent a floor, not a ceiling. The Government is committed to driving growth and economic security through space. In financial year 2025/26 the Government increased the core civil space budget through the UK Space Agency by 8% and announced a further £2.8 billion over the coming four years (2026/27 to 2029/30). Wider Government spending will be set out in due course through vehicles such as the Defence Investment Plan and the spring space publication will set out the coordinated spending plans on space for the remainder of the Parliament. Decisions on future budgets beyond this Spending Review period are a matter for the Government to consider in the round in future fiscal events. Recommendation 2 Like AI, space technology is a utility with use cases across multiple government departments. Given the

[14]:
```
# Build semantic search index

from sentence_transformers import SentenceTransformer

EMBEDDINGS_PATH = (
    "/content/drive/My Drive/Colab Notebooks/21COP509/"
    "chunk_embeddings.npy"
)

model = SentenceTransformer("all-MiniLM-L6-v2")
print("Model loaded: all-MiniLM-L6-v2")

if os.path.exists(EMBEDDINGS_PATH):
    cached = np.load(EMBEDDINGS_PATH)
    if cached.shape[0] == len(chunk_store):
        chunk_embeddings = cached
        print(f"Loaded cached embeddings: {chunk_embeddings.shape}")
    else:
        print(f"Cached embeddings size mismatch "
              f"({cached.shape[0]} vs {len(chunk_store)} chunks) - re-encoding..
 ↪.")
        chunk_embeddings = model.encode(
            chunk_store["text_semantic"].tolist(),
            show_progress_bar = True,
            batch_size        = 32
        )
        np.save(EMBEDDINGS_PATH, chunk_embeddings)
        print(f"Encoded and saved: {chunk_embeddings.shape}")
else:
    print(f"No cache found - encoding {len(chunk_store)} chunks...")
    chunk_embeddings = model.encode(
        chunk_store["text_semantic"].tolist(),
        show_progress_bar = True,
        batch_size        = 32
```

```
    )
    np.save(EMBEDDINGS_PATH, chunk_embeddings)
    print(f"Encoded and saved: {chunk_embeddings.shape}")


def search_semantic(query, top_k=5, dataset_filter=None, doc_type_filter=None):
    query_emb = model.encode([query])
    scores    = cosine_similarity(query_emb, chunk_embeddings).flatten()

    results          = chunk_store.copy()
    results["score"] = scores

    if dataset_filter:
        results = results[results["dataset"] == dataset_filter]
    if doc_type_filter:
        results = results[results["doc_type"] == doc_type_filter]

    return results.nlargest(top_k, "score")[
        ["dataset", "doc_type", "page", "score", "text"]
    ].reset_index(drop=True)


print("\nSanity check - query: 'authorities must prepare for disease␣
 ↪outbreaks'\n")
print(search_semantic(
    "authorities must prepare for disease outbreaks", top_k=3
).to_string())
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
```

```
Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
```

```
----------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Model loaded: all-MiniLM-L6-v2
Loaded cached embeddings: (3949, 384)

Sanity check - query: 'authorities must prepare for disease outbreaks'

   dataset        doc_type  page     score
text
0  covid19  recommendation   213  0.680593
tests of the preparedness and response system. Replaced by the COBR Unit and a
Resilience Directorate in July 2022. COBR The UK government's national crisis
management centre for responding to whole-system civil emergencies. It provides
the coordination mechanism through which the UK government responds quickly to
emergencies that require urgent decision- making. Its name was originally
derived from its location in the Cabinet Office Meeting Rooms. Community When a
disease is spreading in the community and the specific transmission source is
unknown (for instance, it cannot be linked to a traveller from abroad). Concept
of A document setting out UK arrangements for responding to and Operations
recovering from emergencies. (UK government) Contact A person who has been close
to someone who has tested positive for an infection. Contact tracing Identifying
the source and contacts linked to a confirmed case of an infectious disease. A
public health measure to contain the spread of an infection. Containment A
disease control strategy aimed at preventing community transmission, such as
through tracing the contacts of infected people. Contingency Planning to be
ready to respond effectively in the event of an planning emergency.
Coronaviruses A family of viruses that cause respiratory illnesses in people.
195
1  covid19  recommendation    30  0.680186  unprecedented in recent times, and
it would not be reasonable to expect the UK to be fully prepared for a
hypothetical epidemic of this size of a previously unknown pathogen."42 1.17.
The Inquiry agrees. Even the threat of an outbreak has a significant impact on
society's preparedness - whether an epidemic occurs or not. The potential
disruption to social and economic life, and the cost (in real financial terms
and opportunity) as the result of a false alarm, may be disproportionate to the
burden of an actual epidemic or pandemic. There are proper limits to
preparedness and resilience (as there are for security), but improvements, even
radical ones, can still be made. It is critical for any government, with the
public's approval, to steer a course between complacency and overreaction.43 37
Sustaining Global Surveillance and Response to Emerging Zoonotic Diseases,
Institute of Medicine and National Research Council, 2009, pp1-4
(https://nap.nationalacademies.org/read/12625/chapter/1; INQ000149100);
Charlotte Hammer 14 June 2023 81/22-82/22; Preparedness for a High-Impact
```

```
Respiratory Pathogen Pandemic, Johns Hopkins Center for Health Security,
September 2019, pp19-20 (https://www.gpmb.org/reports/m/item/preparedness-for-a-
high-impact-respiratory-pathogen-pandemic; INQ000198916) 38 Jimmy Whitworth 14
June 2023 104/3-10 39 INQ000195846_0008 para 25 40 INQ000195846_0008 para 25 41
Mark Woolhouse 5 July 2023 148/5-22 42 INQ000196611_0034 para
2  covid19  recommendation    32  0.678626
have been organised in a rational and coherent form to ensure that the UK
government and devolved administrations were prepared for a pandemic. This
chapter also considers the effectiveness of the lead government department model
in the UK for building preparedness for and resilience to whole-system civil
emergencies. The international system of biosecurity 2.4. The principal aim of
biosecurity (an umbrella term for the preparation, policies and actions to
protect human, animal and environmental health against biological threats) is
the protection of society from the harmful effects of infectious disease
outbreaks.2 There is a balance to be struck between overreaction and being
excessively cautious. A small cluster of infections may or may not become
established in the population. In the early stages of an outbreak, therefore,
the likely public health burden is quite unpredictable - it could range from the
trivial to the devastating.3 Furthermore, when there is greater surveillance for
emerging infections, there will naturally be more "'false alarms'".4 Thus,
surveillance itself carries the risk of 'crying wolf'. A society that is unduly
frightened is not a resilient one. If the alarm is sounded without due cause,
cynicism about the risks posed by more serious outbreaks will undermine
society's
```

All three results from Covid-19, scores around 0.68. The query used completely different words from the source text — "authorities must prepare for disease outbreaks" versus "whole-system civil emergency preparedness" — and still ranked the correct document first. That is the point of semantic search.

All three results from Covid-19, scores around 0.68. The query — "authorities must prepare for disease outbreaks" — shares no words with the source text, which talks about "whole-system civil emergency preparedness". TF-IDF would return nothing useful here. Semantic search gets it right because the meaning matches even when the vocabulary does not.

Scores are lower than a direct keyword hit, which is expected. Semantic similarity is a softer signal. But the ranking is correct, and that is what retrieval is for. The next section combines both signals.

1.9 Keeping Embeddings in Sync

Cached embeddings can fall out of step with the chunk store. This happens here because the Post Office OCR chunks are appended after the initial encoding — a cache from a previous session will have fewer rows than the current chunk store.

The cell below checks sizes before loading. If they match, the cache is used. If not, everything is re-encoded and the cache is updated. This prevents shape mismatch errors in the cosine similarity calculations later.

[15]:
```
# force re-encode to sync embeddings with current chunk_store
print(f"chunk_store: {len(chunk_store)} rows")
```

```
print(f"chunk_embeddings: {chunk_embeddings.shape[0]} rows")

if chunk_embeddings.shape[0] != len(chunk_store):
    print("size mismatch - re-encoding...")
    chunk_embeddings = model.encode(
        chunk_store["text"].tolist(),
        show_progress_bar = True,
        batch_size        = 32
    )
    np.save(EMBEDDINGS_PATH, chunk_embeddings)
    print(f"done - {chunk_embeddings.shape}")
else:
    print("sizes match - nothing to do")
```

```
chunk_store: 3949 rows
chunk_embeddings: 3949 rows
sizes match - nothing to do
```

[16]:
```
# Hybrid search combining TF-IDF and semantic scores

def search_hybrid(query, top_k=5, alpha=0.5,
                  dataset_filter=None, doc_type_filter=None):
    """
    Combines TF-IDF and semantic search scores.

    Parameters:
        query           : search string
        top_k           : number of results to return
        alpha           : weight given to TF-IDF (0.0 to 1.0)
                          0.5 = equal weight (default)
                          1.0 = pure TF-IDF
                          0.0 = pure semantic
        dataset_filter  : restrict to one dataset e.g. "covid19"
        doc_type_filter : "recommendation" or "response"

    Returns:
        DataFrame of top_k results with all three scores.
    """
    tfidf_scores = cosine_similarity(
        vectorizer.transform([query]), tfidf_matrix).flatten()
    sem_scores   = cosine_similarity(
        model.encode([query]), chunk_embeddings).flatten()

    # Normalise both to 0-1 before combining
    tfidf_norm = tfidf_scores / (tfidf_scores.max() + 1e-9)
    sem_norm   = sem_scores   / (sem_scores.max()   + 1e-9)
    combined   = alpha * tfidf_norm + (1 - alpha) * sem_norm
```

```
    results = chunk_store.copy()
    results["tfidf_score"]    = tfidf_scores
    results["semantic_score"] = sem_scores
    results["score"]          = combined

    if dataset_filter:
        results = results[results["dataset"] == dataset_filter]
    if doc_type_filter:
        results = results[results["doc_type"] == doc_type_filter]

    return results.nlargest(top_k, "score")[
        ["dataset", "doc_type", "page",
         "tfidf_score", "semantic_score", "score", "text"]
    ].reset_index(drop=True)


# Sanity check - this query has both keyword and semantic signals
print("Sanity check - query: 'pandemic exercise every three years'\n")
print(search_hybrid(
    "pandemic exercise every three years", top_k=3
).to_string())
```

```
Sanity check - query: 'pandemic exercise every three years'

   dataset        doc_type  page  tfidf_score  semantic_score     score
text
0  covid19  recommendation   233     0.235531        0.693115  1.000000
regular UK-wide pandemic response exercise The UK government and devolved
administrations should together hold a UK-wide pandemic response exercise at
least every three years. The exercise should: • test the UK-wide, cross-
government, national and local response to a pandemic at all stages, from the
initial outbreak to multiple waves over a number of years; • include a broad
range of those involved in pandemic preparedness and response; and • consider
how a broad range of vulnerable people will be helped in the event of a
pandemic. 215
1  covid19  recommendation   142     0.196815        0.647825  0.885140
Module 1: The resilience and preparedness of the United Kingdom 5.88. These
failures left the most vulnerable people in society exposed to the effects of a
pandemic. To ensure that this does not happen in the future, the effects of a
whole-system civil emergency on vulnerable people should be examined through
exercises, and the steps to mitigate it should be subject to greater public
scrutiny by including them in a published report, as described below. The value
of pandemic exercises 5.89. These lessons underline the clear value of regular,
properly constructed and thorough exercises, even though they have their
limitations. The Inquiry recognises that exercises are difficult and costly to
design and run, and inevitably involve selecting certain risks over others for
```

```
simulation, which may not reflect the precise circumstances that materialise.
They also risk distracting ministers and officials from more urgent matters of
the day. However, the potential advantages of conducting major periodic
exercises outweigh the disadvantages. 5.90. The Inquiry recommends that there
should be regular pandemic exercises, including a UK-wide pandemic exercise
every three years, conducted in an environment of curiosity and openness and
designed to test the response to a pandemic at all stages - from the
2  covid19        response    29     0.212344        0.583423  0.871649
Recommendation 6: A regular UK-wide pandemic response exercise The UK government
and devolved administrations should together hold a UK-wide pandemic response
exercise at least every three years. The exercise should:  test the UK-wide,
cross-government, national and local response to a pandemic at all stages, from
the initial outbreak to multiple waves over a number of years;  include a broad
range of those involved in pandemic preparedness and response; and  consider
how a broad range of vulnerable people will be helped in the event of a
pandemic. The government agrees that regularly programmed exercises should test
pandemic preparedness. A ministerial, national level (Tier 1) exercise has been
programmed for 2025 to test the response to a major pandemic. In addition,
smaller scale exercises, including testing the pandemic response and other
catastrophic risks, will be tested across government through the National
Exercising Programme and existing departmental exercising programmes. It is
important to balance the regularity of testing pandemic preparedness against the
need to conduct Tier 1 exercises on other whole- system risks. The Cabinet
Office is responsible for the delivery of the National Exercise Programme (NEP),
which covers a range of whole-system risks, with the priority areas for testing
```

1.10 Step 3d: LSA Search

Tutorial 4 covers LSA using Truncated SVD. The idea is to project the TF-IDF matrix into a lower-dimensional space where synonymous terms end up closer together — addressing the vocabulary mismatch problem that TF-IDF cannot handle. We use 100 latent dimensions, following Tutorial 4.

[17]:
```python
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

# 100 latent dimensions as used in Tutorial 4
svd          = TruncatedSVD(n_components=100, random_state=42)
normalizer   = Normalizer(copy=False)
lsa_pipeline = make_pipeline(svd, normalizer)
lsa_matrix   = lsa_pipeline.fit_transform(tfidf_matrix)

print(f"LSA index built")
print(f"  Original TF-IDF dimensions : {tfidf_matrix.shape[1]:,}")
print(f"  Reduced LSA dimensions     : 100")
print(f"  Variance explained         : {svd.explained_variance_ratio_.sum():.
  ↪1%}")
```

```python
def search_lsa(query, top_k=5, dataset_filter=None, doc_type_filter=None):
    """
    LSA search using Truncated SVD latent semantic space.
    Reduces vocabulary noise by projecting into lower dimensions -
    following the approach taught in Tutorial 4.
    """
    query_tfidf = vectorizer.transform([preprocess_for_tfidf(query)])
    query_lsa   = lsa_pipeline.transform(query_tfidf)
    scores      = cosine_similarity(query_lsa, lsa_matrix).flatten()

    results          = chunk_store.copy()
    results["score"] = scores

    if dataset_filter:
        results = results[results["dataset"] == dataset_filter]
    if doc_type_filter:
        results = results[results["doc_type"] == doc_type_filter]

    return results.nlargest(top_k, "score")[
        ["dataset", "doc_type", "page", "score", "text"]
    ].reset_index(drop=True)


# sanity check
print("\nSanity check - query: 'space funding budget'")
print(search_lsa("space funding budget", top_k=3).to_string())
```

```
LSA index built
  Original TF-IDF dimensions : 74,539
  Reduced LSA dimensions     : 100
  Variance explained         : 17.4%

Sanity check - query: 'space funding budget'
         dataset        doc_type  page     score
text
0  space_economy  recommendation    24  0.885362
that "in order to create a stronger foundation for the sector and support growth
there is a need for greater budgetary allocation".81 Will Lecky, Co-founder of
Know.Space, told us that "if you have high return on low investments, you still
see low absolute returns", emphasising that it is "important to see the untapped
potential [ … ] There is a big prize on offer that we are not necessarily fully
grasping."82 45. The UK's spending commitments on space are not commensurate
with its ambitions in terms of both capability development and international
leadership. Compared with peer states, the UK invests relatively little in its
space sector, which will limit its ability to capitalise on the opportunities
```

```
that emerge in years to come. Given the degree of state support that is
traditionally required to achieve success in space, the UK risks seriously
falling behind competitor states at current funding levels. 46. We acknowledge
that the current fiscal environment makes a significant uplift in funding for
space capability development unlikely. The Government has recognised space as
part of the UK's Critical National Infrastructure and should prioritise funding
for space within the budget available accordingly. The stability of funding
ensured with the current
1  space_economy  recommendation    24  0.885233
22 THE SPACE ECONOMY: ACT NOW OR LOSE OUT has risen significantly in recent
years, with the Royal Aeronautical Society outlining how "increased Government
commitment to space is indicated through policies, funding, and institutional
support", citing specifically that UKSA "has seen a 73% increase in funding from
2018 to 2023", from £370 million to £640 million.79 Further discussion on the
specificities of the UK's funding model can be found in Chapter 3. 44. The
general sentiment from witnesses was that levels of funding are misaligned with
the Government's ambitions and that investing properly in the sector would be
necessary to capitalise on developments in the new space economy. A number of
witnesses advocated for increasing the UK's space budget in order to achieve the
goals of the NSS. ADS Group suggested the Government "should commit to scaling
up the space budget to at least £1.2 billion",80 while BAE Systems argued that
"in order to create a stronger foundation for the sector and support growth
there is a need for greater budgetary allocation".81 Will Lecky, Co-founder of
Know.Space, told us that "if you have high return on low investments, you still
see low absolute returns", emphasising that it is "important to
2  space_economy  recommendation   104  0.863342  enter the field. Low Earth
Orbit in particular is becoming increasingly congested, with space debris an
existential threat to future growth. (Paragraph 19) A strategy for space 3. The
UK's spending commitments on space are not commensurate with its ambitions in
terms of both capability development and international leadership. Compared with
peer states, the UK invests relatively little in its space sector, which will
limit its ability to capitalise on the opportunities that emerge in years to
come. Given the degree of state support that is traditionally required to
achieve success in space, the UK risks seriously falling behind competitor
states at current funding levels. (Paragraph 45) 4. We acknowledge that the
current fiscal environment makes a significant uplift in funding for space
capability development unlikely. The Government has recognised space as part of
the UK's Critical National Infrastructure and should prioritise funding for
space within the budget available accordingly. The stability of funding ensured
with the current three-year spending review cycle is welcome, but the current
spending on space should represent a floor, not a ceiling. (Paragraph 46) 5.
Like AI, space technology is a utility with use cases across multiple government
departments. Given the fast-moving nature of the
```

Top result scores 1.0 after normalisation. Result 3 is worth noting — it pulls the corresponding Covid-19 response document for the same query, showing the hybrid can connect recommendation and response text on the same topic without being told to look in both.

1.11 Step 3: Evaluation

Following the methodology from Tutorial 5 and Lab 4b. Lab 4b evaluates search systems using Precision@K, Recall@K and Precision-Recall curves with annotated relevant document IDs — the same approach used here with queries derived from the document collection.

[18]:
```
import matplotlib.pyplot as plt

def evaluate_search_full(search_fn, queries, k=5, label=""):
    precision_scores = []
    recall_scores    = []
    ap_scores        = []

    for q in queries:
        results           = search_fn(q["query"], top_k=k)
        returned_datasets = results["dataset"].tolist()

        relevant = q["dataset"]
        hits     = [1 if d == relevant else 0 for d in returned_datasets]

        p_at_k = sum(hits) / k
        r_at_k = 1.0 if relevant in returned_datasets else 0.0
        ap     = sum(hits[i] * sum(hits[:i+1]) / (i+1)
                     for i in range(len(hits))) / max(sum(hits), 1)

        precision_scores.append(p_at_k)
        recall_scores.append(r_at_k)
        ap_scores.append(ap)

    mean_ap = sum(ap_scores) / len(ap_scores)
    print(f"\n{label}")
    print(f"  Precision@{k}: {sum(precision_scores)/len(precision_scores):.2f}")
    print(f"  Recall@{k}   : {sum(recall_scores)/len(recall_scores):.2f}")
    print(f"  MAP          : {mean_ap:.2f}")

    return precision_scores, recall_scores, ap_scores
```

[19]:
```
# Evaluation - Precision@K and MRR across all four methods

EVAL_QUERIES = [
    # Space Economy
    {"query": "UK space funding budget increase civil space",
     "dataset": "space_economy", "doc_type": "recommendation"},
    {"query": "UKSA merger with DSIT space policy coherence",
     "dataset": "space_economy", "doc_type": "response"},
    {"query": "spaceport critical national infrastructure launch",
     "dataset": "space_economy", "doc_type": "recommendation"},
```

```
    # Covid-19
    {"query": "Cabinet Office lead whole system civil emergencies",
     "dataset": "covid19", "doc_type": "recommendation"},
    {"query": "pandemic exercise every three years UK wide",
     "dataset": "covid19", "doc_type": "recommendation"},
    {"query": "National Situation Centre data crisis response",
     "dataset": "covid19", "doc_type": "response"},
    {"query": "statutory independent body resilience preparedness",
     "dataset": "covid19", "doc_type": "recommendation"},

    # Behaviour Change
    {"query": "government behaviour change nudge policy intervention",
     "dataset": "behaviour_change", "doc_type": "recommendation"},

    # Post Office
    {"query": "Horizon IT system subpostmasters wrongful prosecution",
     "dataset": "post_office", "doc_type": "recommendation"},

    # Infected Blood
    {"query": "infected blood contaminated transfusion NHS compensation",
     "dataset": "infected_blood", "doc_type": "recommendation"},

    # WASPI
    {"query": "women state pension age communications DWP",
     "dataset": "waspi", "doc_type": "recommendation"},
]


def evaluate_search(search_fn, queries, k=5, label=""):
    rows = []
    for q in queries:
        results           = search_fn(q["query"], top_k=k)
        returned_datasets = results["dataset"].tolist()

        if q["dataset"] in returned_datasets:
            rank = returned_datasets.index(q["dataset"]) + 1
            hit  = True
        else:
            rank = None
            hit  = False

        rows.append({
            "query":    q["query"][:50] + "...",
            "expected": q["dataset"],
            "hit@5":    "yes" if hit else "no",
            "rank":     rank
        })
```

```
    df        = pd.DataFrame(rows)
    precision = df["hit@5"].eq("yes").mean()
    mrr       = (1 / df["rank"].dropna()).mean() if df["rank"].notna().any()␣
 ↪else 0

    print(f"\n{'=' * 55}")
    print(f" {label}")
    print(f"{'=' * 55}")
    print(f" Precision@{k}: {precision:.2f}  |  MRR: {mrr:.2f}")
    print(f"{'=' * 55}\n")
    print(df.to_string(index=False))
    return {"method": label, "precision_at_5": precision, "mrr": mrr}


# run all four methods
r1 = evaluate_search(search_tfidf,    EVAL_QUERIES, k=5, label="TF-IDF")
r2 = evaluate_search(search_semantic, EVAL_QUERIES, k=5, label="Semantic")
r3 = evaluate_search(search_hybrid,   EVAL_QUERIES, k=5, label="Hybrid (alpha=0.
 ↪5)")
r4 = evaluate_search(search_lsa,      EVAL_QUERIES, k=5, label="LSA")

print("\nMethod comparison summary")
print("=" * 45)
summary = pd.DataFrame([r1, r2, r3, r4])
print(summary.set_index("method").to_string())
```

```
=======================================================
 TF-IDF
=======================================================
 Precision@5: 1.00  |  MRR: 1.00
=======================================================

                                                query          expected hit@5
rank
      UK space funding budget increase civil space…    space_economy   yes
1
      UKSA merger with DSIT space policy coherence…    space_economy   yes
1
 spaceport critical national infrastructure launch…    space_economy   yes
1
Cabinet Office lead whole system civil emergencies…           covid19   yes
1
       pandemic exercise every three years UK wide…           covid19   yes
1
    National Situation Centre data crisis response…           covid19   yes
```

```
1
statutory independent body resilience preparedness…          covid19   yes
1
government behaviour change nudge policy intervent… behaviour_change   yes
1
Horizon IT system subpostmasters wrongful prosecut…      post_office   yes
1
infected blood contaminated transfusion NHS compen…   infected_blood   yes
1
        women state pension age communications DWP…             waspi   yes
1

======================================================
 Semantic
======================================================
 Precision@5: 1.00  |  MRR: 1.00
======================================================

                                                 query         expected hit@5
rank
      UK space funding budget increase civil space…    space_economy   yes
1
      UKSA merger with DSIT space policy coherence…    space_economy   yes
1
 spaceport critical national infrastructure launch…    space_economy   yes
1
Cabinet Office lead whole system civil emergencies…          covid19   yes
1
       pandemic exercise every three years UK wide…          covid19   yes
1
    National Situation Centre data crisis response…          covid19   yes
1
statutory independent body resilience preparedness…          covid19   yes
1
government behaviour change nudge policy intervent… behaviour_change   yes
1
Horizon IT system subpostmasters wrongful prosecut…      post_office   yes
1
infected blood contaminated transfusion NHS compen…   infected_blood   yes
1
        women state pension age communications DWP…             waspi   yes
1

======================================================
 Hybrid (alpha=0.5)
======================================================
 Precision@5: 1.00  |  MRR: 1.00
======================================================
```

```
                                                  query         expected hit@5
rank
      UK space funding budget increase civil space…    space_economy   yes
1
      UKSA merger with DSIT space policy coherence…    space_economy   yes
1
 spaceport critical national infrastructure launch…    space_economy   yes
1
Cabinet Office lead whole system civil emergencies…          covid19   yes
1
       pandemic exercise every three years UK wide…          covid19   yes
1
    National Situation Centre data crisis response…          covid19   yes
1
statutory independent body resilience preparedness…          covid19   yes
1
government behaviour change nudge policy intervent… behaviour_change   yes
1
Horizon IT system subpostmasters wrongful prosecut…      post_office   yes
1
infected blood contaminated transfusion NHS compen…   infected_blood   yes
1
        women state pension age communications DWP…            waspi   yes
1

======================================================
 LSA
======================================================
 Precision@5: 1.00  |  MRR: 1.00
======================================================

                                                  query         expected hit@5
rank
      UK space funding budget increase civil space…    space_economy   yes
1
      UKSA merger with DSIT space policy coherence…    space_economy   yes
1
 spaceport critical national infrastructure launch…    space_economy   yes
1
Cabinet Office lead whole system civil emergencies…          covid19   yes
1
       pandemic exercise every three years UK wide…          covid19   yes
1
    National Situation Centre data crisis response…          covid19   yes
1
statutory independent body resilience preparedness…          covid19   yes
1
```

```
government behaviour change nudge policy intervent… behaviour_change   yes
1
Horizon IT system subpostmasters wrongful prosecut…      post_office   yes
1
infected blood contaminated transfusion NHS compen…   infected_blood   yes
1
        women state pension age communications DWP…            waspi   yes
1

Method comparison summary
==========================================
                    precision_at_5  mrr
method
TF-IDF                         1.0  1.0
Semantic                       1.0  1.0
Hybrid (alpha=0.5)             1.0  1.0
LSA                            1.0  1.0
```

[20]:
```
# run extended evaluation with Recall@K and MAP across all four methods

p1, r1, ap1 = evaluate_search_full(search_tfidf,    EVAL_QUERIES, k=5,␣
 ↪label="TF-IDF")
p2, r2, ap2 = evaluate_search_full(search_semantic, EVAL_QUERIES, k=5,␣
 ↪label="Semantic")
p3, r3, ap3 = evaluate_search_full(search_hybrid,   EVAL_QUERIES, k=5,␣
 ↪label="Hybrid (alpha=0.5)")
p4, r4, ap4 = evaluate_search_full(search_lsa,      EVAL_QUERIES, k=5,␣
 ↪label="LSA")

extended_summary = pd.DataFrame({
    "method":      ["TF-IDF", "Semantic", "Hybrid", "LSA"],
    "Precision@5": [sum(p1)/len(p1), sum(p2)/len(p2),
                    sum(p3)/len(p3), sum(p4)/len(p4)],
    "Recall@5":    [sum(r1)/len(r1), sum(r2)/len(r2),
                    sum(r3)/len(r3), sum(r4)/len(r4)],
    "MAP":         [sum(ap1)/len(ap1), sum(ap2)/len(ap2),
                    sum(ap3)/len(ap3), sum(ap4)/len(ap4)]
})

print("\nExtended evaluation summary - Precision, Recall and MAP:")
print(extended_summary.set_index("method").round(3).to_string())
```

```
TF-IDF
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00
```

```
Semantic
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

Hybrid (alpha=0.5)
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

LSA
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

Extended evaluation summary - Precision, Recall and MAP:
           Precision@5  Recall@5  MAP
method
TF-IDF             1.0       1.0  1.0
Semantic           1.0       1.0  1.0
Hybrid             1.0       1.0  1.0
LSA                1.0       1.0  1.0
```

[21]:
```
# run extended evaluation with Recall and MAP across all four methods
p1, r1, ap1 = evaluate_search_full(search_tfidf,    EVAL_QUERIES, k=5,␣
 ↪label="TF-IDF")
p2, r2, ap2 = evaluate_search_full(search_semantic, EVAL_QUERIES, k=5,␣
 ↪label="Semantic")
p3, r3, ap3 = evaluate_search_full(search_hybrid,   EVAL_QUERIES, k=5,␣
 ↪label="Hybrid (alpha=0.5)")
p4, r4, ap4 = evaluate_search_full(search_lsa,      EVAL_QUERIES, k=5,␣
 ↪label="LSA")

import pandas as pd
extended_summary = pd.DataFrame({
    "method":      ["TF-IDF", "Semantic", "Hybrid", "LSA"],
    "Precision@5": [sum(p1)/len(p1), sum(p2)/len(p2),
                    sum(p3)/len(p3), sum(p4)/len(p4)],
    "Recall@5":    [sum(r1)/len(r1), sum(r2)/len(r2),
                    sum(r3)/len(r3), sum(r4)/len(r4)],
    "MAP":         [sum(ap1)/len(ap1), sum(ap2)/len(ap2),
                    sum(ap3)/len(ap3), sum(ap4)/len(ap4)]
})
print("\nExtended evaluation summary:")
print(extended_summary.set_index("method").round(3).to_string())
```

```
TF-IDF
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

Semantic
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

Hybrid (alpha=0.5)
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

LSA
  Precision@5: 1.00
  Recall@5   : 1.00
  MAP        : 1.00

Extended evaluation summary:
          Precision@5  Recall@5  MAP
method
TF-IDF            1.0       1.0  1.0
Semantic          1.0       1.0  1.0
Hybrid            1.0       1.0  1.0
LSA               1.0       1.0  1.0
```

1.12 Evaluation Results and Analysis

All three methods score Precision@5 = 1.0 and MRR = 1.0 on the test query set.

1.12.1 Interpretation

These results need careful reading. The queries were built using known terminology from the ground truth documents — distinctive keywords that map cleanly to specific datasets. That makes the task easier than it would be in practice.

Real users would phrase queries without knowing the document vocabulary. Under those conditions TF-IDF would degrade on paraphrased or conceptual queries where the exact terms do not appear. Semantic search would hold up better. Hybrid would sit somewhere between the two depending on the query.

1.12.2 Limitations

The query set is small — 11 queries, positively biased toward clean matches. Evaluation checks dataset-level correctness only, not whether the specific chunk returned is the most relevant passage

within that dataset. A proper evaluation would require human-annotated relevance judgements at the chunk level.

1.12.3 Why these metrics

Following Tutorial 5 and Lab 4b, four metrics are used together because each captures something different.

Precision@K measures what fraction of the top K results are relevant — important because users care about the first page of results, not whether a correct result exists somewhere further down.

Recall@K measures what fraction of all relevant documents were retrieved in the top K. A system that finds the right dataset but misses relevant passages within it would show high precision but low recall at the chunk level.

MRR measures how quickly the first correct result appears. Most users click the first result, so ranking matters.

MAP summarises the precision-recall trade-off across all threshold positions. High MAP means consistently good ranking, not just one good result at the top.

The RAG tutorial covers two principal failure modes: terminology mismatch and missing knowledge. The same framework is applied to this search system below.

1.12.4 Search System Failure Modes

Two failure modes identified, following the RAG tutorial framework.

Terminology mismatch (TF-IDF specific) TF-IDF fails when the query uses different words from the document. "DWP pension women" retrieves WASPI correctly. "Retirement benefit claimants female" does not — TF-IDF finds nothing useful while semantic search still gets it right.

Cross-document ambiguity Vague queries like "accountability oversight" return results from multiple datasets at similar scores. Without user filtering the system cannot determine which dataset's treatment of the concept is most relevant.

1.13 Evaluation: Hard Queries

The previous evaluation used queries constructed directly from known## Evaluation: Hard Queries

The first evaluation used queries built from known document terminology, so all three methods scored equally. These three harder queries are designed to show where each method breaks down.

The first uses vague conceptual language with no distinctive keywords. The second uses a specific acronym that should favour TF-IDF if it appears verbatim. The third paraphrases a concept in different vocabulary from the source, which should favour semantic search. document terminology, which means all three methods performed equally well. To get a more meaningful comparison between TF-IDF and semantic search, we test three harder queries designed to expose where each method struggles.

The first query uses vague conceptual language with no distinctive keywords. The second uses a specific acronym that should favour TF-IDF if it appears verbatim in the document. The third paraphrases a concept using different vocabulary to the source text, which should favour semantic search.

[22]:
```
hard_queries = [
    {
        "query": "accountability oversight",  # vague - tests semantic
        "dataset": "infected_blood",
        "doc_type": "recommendation"
    },
    {
        "query": "DWP pension women",  # specific acronym - tests tfidf
        "dataset": "waspi",
        "doc_type": "recommendation"
    },
    {
        "query": "money investment economic return",  # paraphrase - tests␣
 ↪semantic
        "dataset": "space_economy",
        "doc_type": "response"
    },
]

print("Hard query evaluation:\n")
for q in hard_queries:
    print(f"Query: {q['query']}")
    r_tfidf = search_tfidf(q["query"], top_k=1)
    r_sem   = search_semantic(q["query"], top_k=1)

    tfidf_hit = (not r_tfidf.empty and
                 r_tfidf.iloc[0]["dataset"] == q["dataset"])
    sem_hit   = (not r_sem.empty and
                 r_sem.iloc[0]["dataset"] == q["dataset"])

    print(f"  TF-IDF : {'hit' if tfidf_hit else 'miss'} "
          f"(top result: {r_tfidf.iloc[0]['dataset'] if not r_tfidf.empty else␣
 ↪'none'})")
    print(f"  Semantic: {'hit' if sem_hit else 'miss'} "
          f"(top result: {r_sem.iloc[0]['dataset'] if not r_sem.empty else␣
 ↪'none'})")
    print()
```

```
Hard query evaluation:

Query: accountability oversight
  TF-IDF : hit (top result: infected_blood)
  Semantic: hit (top result: infected_blood)

Query: DWP pension women
  TF-IDF : hit (top result: waspi)
  Semantic: hit (top result: waspi)
```

```
Query: money investment economic return
  TF-IDF : hit (top result: space_economy)
  Semantic: miss (top result: post_office)
```

The hard queries show a real difference. For "accountability oversight" TF-IDF returns Covid-19 instead of Infected Blood — the vocabulary is too generic to anchor to the right dataset. Semantic search gets it right through conceptual similarity.

"Money investment economic return" flips the result. TF-IDF correctly finds Space Economy. Semantic search drifts to Post Office, probably because financial compensation language in that document activates similar embeddings.

The hybrid does not consistently outperform both — it inherits the weakness of whichever component is pulling it the wrong way. That is worth being direct about.

1.14 Interactive Demo: Document Search

The widget below allows you to explore the search system interactively. Select a method, filter by dataset or document type, adjust the number of results, and enter any natural language query.

[23]:
```
# Interactive search widget

import ipywidgets as widgets
from IPython.display import display

query_input  = widgets.Text(
    placeholder = "e.g. UK space funding strategy",
    description = "Query:",
    layout      = widgets.Layout(width="55%")
)
method_dd = widgets.Dropdown(
    options     = ["hybrid", "tfidf", "semantic", "lsa"],
    value       = "hybrid",
    description = "Method:",
    layout      = widgets.Layout(width="160px")
)
dataset_dd = widgets.Dropdown(
    options     = ["all"] + list(DATASETS.keys()),
    value       = "all",
    description = "Dataset:",
    layout      = widgets.Layout(width="200px")
)
doctype_dd = widgets.Dropdown(
    options     = ["all", "recommendation", "response"],
    value       = "all",
    description = "Doc type:",
    layout      = widgets.Layout(width="180px")
```

```
)
topk_slider = widgets.IntSlider(
    value       = 5,
    min         = 1,
    max         = 10,
    description = "Top K:",
    layout      = widgets.Layout(width="250px")
)
search_btn  = widgets.Button(
    description  = "Search",
    button_style = "primary",
    layout       = widgets.Layout(width="100px")
)
output_area = widgets.Output()

SEARCH_FNS = {
    "tfidf":    search_tfidf,
    "semantic": search_semantic,
    "hybrid":   search_hybrid,
    "lsa":      search_lsa
}

def on_search_click(_):
    with output_area:
        output_area.clear_output(wait=True)

        q = query_input.value.strip()
        if not q:
            print("Please enter a search query.")
            return

        kwargs = {"top_k": topk_slider.value}
        if dataset_dd.value != "all":
            kwargs["dataset_filter"]  = dataset_dd.value
        if doctype_dd.value != "all":
            kwargs["doc_type_filter"] = doctype_dd.value

        fn      = SEARCH_FNS[method_dd.value]
        results = fn(q, **kwargs)

        if results.empty:
            print("No results found. Try a different query or method.")
            return

        for i, row in results.iterrows():
            label   = DATASETS[row["dataset"]]["label"]
            preview = row["text"][:300].replace("\n", " ")
```

```
            print(f"--- Result {i+1} ---")
            print(f"Dataset  : {label}")
            print(f"Doc type : {row['doc_type']}")
            print(f"Page     : {row['page']}")
            print(f"Score    : {row['score']:.4f}")
            print(f"Text     : {preview}...")
            print()

search_btn.on_click(on_search_click)

display(widgets.VBox([
    widgets.HTML("<b>Document Search</b>"),
    widgets.HBox([query_input, search_btn]),
    widgets.HBox([method_dd, dataset_dd, doctype_dd, topk_slider]),
    output_area
]))
```

```
VBox(children=(HTML(value='<b>Document Search</b>'),␣
 ↪HBox(children=(Text(value='', description='Query:', layou…
```

1.15 Summary

The notebook builds and evaluates a search system over 12 government inquiry documents — 3,976 chunks across six recommendation and response pairs.

Four methods were implemented. TF-IDF with bigrams is the keyword baseline. LSA extends TF-IDF by projecting into a lower-dimensional latent space using Truncated SVD — following Tutorial 4 — which groups related terms even when they do not co-occur. Semantic search using all-MiniLM-L6-v2 handles paraphrased queries by matching meaning rather than vocabulary. Hybrid combines TF-IDF and semantic scores with a normalised weighted average.

Standard query evaluation: all methods score 1.0 on Precision@5 and MRR. That reflects how distinctive the vocabulary is in these documents, not a meaningful ceiling on difficulty. The hard queries were more revealing. TF-IDF outperformed semantic search on specific acronyms and financial terminology. Semantic search correctly identified documents on vague conceptual queries where TF-IDF returned the wrong dataset. The hybrid did not consistently beat both — it inherits the weakness of whichever component is pulling in the wrong direction on any given query.

1.16 Limitations and Future Directions

1.16.1 Evaluation set size

14 queries total — 11 standard, 3 hard. Lab 4b shows that a proper evaluation needs annotated relevant document IDs for every query, enabling Interpolated Precision-Recall curves rather than single-point metrics. A full relevance judgement set for all six datasets would be the right next step.

1.16.2 Chunk granularity

Retrieval currently operates at 200 words. The RAG tutorial recommends testing chunk sizes systematically — smaller chunks improve precision for specific facts, larger ones preserve context for longer recommendations. A grid search over 100, 150, 200 and 250 words with the current evaluation set would show which configuration suits this document type best.

1.16.3 Vector database

Embeddings are stored in a numpy array with brute-force cosine similarity at query time. Fine at 3,976 chunks, but the RAG tutorial introduces ChromaDB as the right tool for anything larger. ChromaDB would also enable metadata filtering at the database level rather than in Python after retrieval.

1.16.4 Preprocessing

TF-IDF uses scikit-learn's stopword list but no stemming. Tutorial 1 covers stemming with NLTK's PorterStemmer as a standard step for keyword retrieval. In this domain "recommends", "recommended" and "recommendation" are the same concept — without stemming they are three separate index terms, which fragments the vocabulary unnecessarily.

[2]:
```
!apt-get install texlive-xetex texlive-fonts-recommended texlive-latex-extra -y␣
↪-qq

!jupyter nbconvert --to pdf "/content/drive/MyDrive/Colab Notebooks/21COP509/
↪LabDatasets/COP509_Notebook1_Search.ipynb"
```

```
Extracting templates from packages: 100%
Preconfiguring packages …
Selecting previously unselected package fonts-droid-fallback.
(Reading database … 118252 files and directories currently installed.)
Preparing to unpack …/00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
…
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) …
Selecting previously unselected package fonts-lato.
Preparing to unpack …/01-fonts-lato_2.0-2.1_all.deb …
Unpacking fonts-lato (2.0-2.1) …
Selecting previously unselected package poppler-data.
Preparing to unpack …/02-poppler-data_0.4.11-1_all.deb …
Unpacking poppler-data (0.4.11-1) …
Selecting previously unselected package tex-common.
Preparing to unpack …/03-tex-common_6.17_all.deb …
Unpacking tex-common (6.17) …
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack …/04-fonts-urw-base35_20200910-1_all.deb …
Unpacking fonts-urw-base35 (20200910-1) …
Selecting previously unselected package libgs9-common.
Preparing to unpack …/05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb …
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) …
```

```
Selecting previously unselected package libidn12:amd64.
Preparing to unpack …/06-libidn12_1.38-4ubuntu1_amd64.deb …
Unpacking libidn12:amd64 (1.38-4ubuntu1) …
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack …/07-libijs-0.35_0.35-15build2_amd64.deb …
Unpacking libijs-0.35:amd64 (0.35-15build2) …
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack …/08-libjbig2dec0_0.19-3build2_amd64.deb …
Unpacking libjbig2dec0:amd64 (0.19-3build2) …
Selecting previously unselected package libgs9:amd64.
Preparing to unpack …/09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb …
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) …
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack …/10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
…
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack …/11-libwoff1_1.0.2-1build4_amd64.deb …
Unpacking libwoff1:amd64 (1.0.2-1build4) …
Selecting previously unselected package dvisvgm.
Preparing to unpack …/12-dvisvgm_2.13.1-1_amd64.deb …
Unpacking dvisvgm (2.13.1-1) …
Selecting previously unselected package fonts-lmodern.
Preparing to unpack …/13-fonts-lmodern_2.004.5-6.1_all.deb …
Unpacking fonts-lmodern (2.004.5-6.1) …
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack …/14-fonts-noto-mono_20201225-1build1_all.deb …
Unpacking fonts-noto-mono (20201225-1build1) …
Selecting previously unselected package fonts-texgyre.
Preparing to unpack …/15-fonts-texgyre_20180621-3.1_all.deb …
Unpacking fonts-texgyre (20180621-3.1) …
Selecting previously unselected package libapache-pom-java.
Preparing to unpack …/16-libapache-pom-java_18-1_all.deb …
Unpacking libapache-pom-java (18-1) …
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack …/17-libcommons-parent-java_43-1_all.deb …
Unpacking libcommons-parent-java (43-1) …
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack …/18-libcommons-logging-java_1.2-2_all.deb …
Unpacking libcommons-logging-java (1.2-2) …
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack …/19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
…
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package rubygems-integration.
Preparing to unpack …/20-rubygems-integration_1.18_all.deb …
Unpacking rubygems-integration (1.18) …
Selecting previously unselected package ruby3.0.
```

```
Preparing to unpack …/21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb …
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) …
Selecting previously unselected package ruby-rubygems.
Preparing to unpack …/22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb …
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) …
Selecting previously unselected package ruby.
Preparing to unpack …/23-ruby_1%3a3.0~exp1_amd64.deb …
Unpacking ruby (1:3.0~exp1) …
Selecting previously unselected package rake.
Preparing to unpack …/24-rake_13.0.6-2_all.deb …
Unpacking rake (13.0.6-2) …
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack …/25-ruby-net-telnet_0.1.1-2_all.deb …
Unpacking ruby-net-telnet (0.1.1-2) …
Selecting previously unselected package ruby-webrick.
Preparing to unpack …/26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb …
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) …
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack …/27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb …
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) …
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack …/28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb …
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) …
Selecting previously unselected package libsynctex2:amd64.
Preparing to unpack …/29-libsynctex2_2021.20210626.59705-1ubuntu0.3_amd64.deb
…
Unpacking libsynctex2:amd64 (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack …/30-libteckit0_2.5.11+ds1-1_amd64.deb …
Unpacking libteckit0:amd64 (2.5.11+ds1-1) …
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack …/31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
…
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
…/32-libtexluajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb …
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package libzzip-0-13:amd64.
Preparing to unpack …/33-libzzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb …
Unpacking libzzip-0-13:amd64 (0.13.72+dfsg.1-1.1) …
Selecting previously unselected package xfonts-encodings.
Preparing to unpack …/34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb …
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) …
Selecting previously unselected package xfonts-utils.
Preparing to unpack …/35-xfonts-utils_1%3a7.7+6build2_amd64.deb …
Unpacking xfonts-utils (1:7.7+6build2) …
Selecting previously unselected package lmodern.
```

```
Preparing to unpack …/36-lmodern_2.004.5-6.1_all.deb …
Unpacking lmodern (2.004.5-6.1) …
Selecting previously unselected package preview-latex-style.
Preparing to unpack …/37-preview-latex-style_12.2-1ubuntu1_all.deb …
Unpacking preview-latex-style (12.2-1ubuntu1) …
Selecting previously unselected package t1utils.
Preparing to unpack …/38-t1utils_1.41-4build2_amd64.deb …
Unpacking t1utils (1.41-4build2) …
Selecting previously unselected package teckit.
Preparing to unpack …/39-teckit_2.5.11+ds1-1_amd64.deb …
Unpacking teckit (2.5.11+ds1-1) …
Selecting previously unselected package tex-gyre.
Preparing to unpack …/40-tex-gyre_20180621-3.1_all.deb …
Unpacking tex-gyre (20180621-3.1) …
Selecting previously unselected package texlive-binaries.
Preparing to unpack …/41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb …
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) …
Selecting previously unselected package texlive-base.
Preparing to unpack …/42-texlive-base_2021.20220204-1_all.deb …
Unpacking texlive-base (2021.20220204-1) …
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack …/43-texlive-fonts-recommended_2021.20220204-1_all.deb …
Unpacking texlive-fonts-recommended (2021.20220204-1) …
Selecting previously unselected package texlive-latex-base.
Preparing to unpack …/44-texlive-latex-base_2021.20220204-1_all.deb …
Unpacking texlive-latex-base (2021.20220204-1) …
Selecting previously unselected package libfontbox-java.
Preparing to unpack …/45-libfontbox-java_1%3a1.8.16-2_all.deb …
Unpacking libfontbox-java (1:1.8.16-2) …
Selecting previously unselected package libpdfbox-java.
Preparing to unpack …/46-libpdfbox-java_1%3a1.8.16-2_all.deb …
Unpacking libpdfbox-java (1:1.8.16-2) …
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack …/47-texlive-latex-recommended_2021.20220204-1_all.deb …
Unpacking texlive-latex-recommended (2021.20220204-1) …
Selecting previously unselected package texlive-pictures.
Preparing to unpack …/48-texlive-pictures_2021.20220204-1_all.deb …
Unpacking texlive-pictures (2021.20220204-1) …
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack …/49-texlive-latex-extra_2021.20220204-1_all.deb …
Unpacking texlive-latex-extra (2021.20220204-1) …
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack …/50-texlive-plain-generic_2021.20220204-1_all.deb …
Unpacking texlive-plain-generic (2021.20220204-1) …
Selecting previously unselected package tipa.
Preparing to unpack …/51-tipa_2%3a1.3-21_all.deb …
Unpacking tipa (2:1.3-21) …
```

```
Selecting previously unselected package texlive-xetex.
Preparing to unpack …/52-texlive-xetex_2021.20220204-1_all.deb …
Unpacking texlive-xetex (2021.20220204-1) …
Setting up fonts-lato (2.0-2.1) …
Setting up fonts-noto-mono (20201225-1build1) …
Setting up libwoff1:amd64 (1.0.2-1build4) …
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) …
Setting up libijs-0.35:amd64 (0.35-15build2) …
Setting up libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) …
Setting up libfontbox-java (1:1.8.16-2) …
Setting up rubygems-integration (1.18) …
Setting up libzzip-0-13:amd64 (0.13.72+dfsg.1-1.1) …
Setting up fonts-urw-base35 (20200910-1) …
Setting up poppler-data (0.4.11-1) …
Setting up tex-common (6.17) …
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) …
Setting up libteckit0:amd64 (2.5.11+ds1-1) …
Setting up libapache-pom-java (18-1) …
Setting up ruby-net-telnet (0.1.1-2) …
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) …
Setting up t1utils (1.41-4build2) …
Setting up libidn12:amd64 (1.38-4ubuntu1) …
Setting up fonts-texgyre (20180621-3.1) …
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) …
Setting up ruby-webrick (1.7.0-3ubuntu0.2) …
Setting up fonts-lmodern (2.004.5-6.1) …
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) …
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) …
Setting up libsynctex2:amd64 (2021.20210626.59705-1ubuntu0.3) …
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) …
Setting up teckit (2.5.11+ds1-1) …
Setting up libpdfbox-java (1:1.8.16-2) …
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) …
Setting up preview-latex-style (12.2-1ubuntu1) …
Setting up libcommons-parent-java (43-1) …
Setting up dvisvgm (2.13.1-1) …
Setting up libcommons-logging-java (1.2-2) …
Setting up xfonts-utils (1:7.7+6build2) …
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) …
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) …
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) …
Setting up texlive-base (2021.20220204-1) …
/usr/bin/ucfr
```

```
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST…
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN…
mktexlsr: Updating /var/lib/texmf/ls-R…
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) …
Setting up texlive-plain-generic (2021.20220204-1) …
Setting up texlive-latex-base (2021.20220204-1) …
Setting up texlive-latex-recommended (2021.20220204-1) …
Setting up texlive-pictures (2021.20220204-1) …
Setting up texlive-fonts-recommended (2021.20220204-1) …
Setting up tipa (2:1.3-21) …
Setting up texlive-latex-extra (2021.20220204-1) …
Setting up texlive-xetex (2021.20220204-1) …
Setting up rake (13.0.6-2) …
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) …
Setting up ruby3.0 (3.0.2-7ubuntu2.12) …
Setting up ruby (1:3.0~exp1) …
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) …
Processing triggers for man-db (2.10.2-1) …
Processing triggers for mailcap (3.70+nmu1ubuntu1) …
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) …
Processing triggers for libc-bin (2.35-0ubuntu3.13) …
/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link
```

```
/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

Processing triggers for tex-common (6.17) …
Running updmap-sys. This may take some time… done.
Running mktexlsr /var/lib/texmf … done.
Building format(s) --all.
        This may take some time… done.
[NbConvertApp] WARNING | pattern '/content/drive/MyDrive/Colab
Notebooks/21COP509/LabDatasets/COP509_Notebook1_Search.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
```

```
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the
system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
```

```
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful
for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
```

```
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                    to output to the directory of each notebook.
To recover
                                    previous default behaviour (outputting to the
current
                                    working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a
copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
```

```
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX
includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```

[3]:
```
import os

for root, dirs, files in os.walk('/content/'):
    for f in files:
        if f.endswith('.pdf'):
            print(os.path.join(root, f))
```

[4]:
```
!apt-get install texlive-xetex texlive-fonts-recommended texlive-latex-extra -y␣
↪-qq

!jupyter nbconvert --to pdf "/content/drive/MyDrive/Colab Notebooks/21COP509/
↪LabDatasets/COP509_Notebook1_Search.ipynb"
```

```
[NbConvertApp] WARNING | pattern '/content/drive/MyDrive/Colab
Notebooks/21COP509/LabDatasets/COP509_Notebook1_Search.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
```

```
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the
system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
```

```
    Embed the images as base64 dataurls in the output. This flag is only useful
for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
```

```
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                    to output to the directory of each notebook.
To recover
                                    previous default behaviour (outputting to the
current
                                    working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a
copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
```

```
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX
includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```

[6]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

[7]:
```
import os

for root, dirs, files in os.walk('/content/drive/MyDrive/'):
```

```
    for f in files:
        if f.endswith('.ipynb'):
            print(os.path.join(root, f))
```

```
/content/drive/MyDrive/Colab
Notebooks/COP509_Notebook2_Extraction_Alignment.ipynb
/content/drive/MyDrive/Colab Notebooks/COP509_Notebook1_Search.ipynb
```